Introduction to Databases

- A database is an organized collection of data
- Allows efficient storage, retrieval, and management of information
- Used in banking, e-commerce, hospitals, and social media
- Managed using Database Management Systems (DBMS)

What is DBMS?

- DBMS is software used to manage databases
- Provides tools for storing, retrieving, and updating data
- Ensures data security and integrity
- Examples: MySQL, PostgreSQL, Oracle Database, SQL Server

Types of Databases

- Relational Databases (SQL)
- Non-Relational Databases (NoSQL)
- *SQL stores structured data in tables*
- *NoSQL stores flexible data like documents or key-value pairs*

SQL vs NoSQL

SQL Databases:

- Structured schema
- Uses SQL queries
- Example: MySQL

NoSQL Databases:

- Flexible schema
- Highly scalable
- Example: DynamoDB

ACID Properties in Databases

- **Atomicity** – All operations succeed or none
- **Consistency** – Data remains valid after transactions
- **Isolation** – Transactions do not interfere with each other
- **Durability** – Committed data remains permanent

Introduction to DynamoDB

- DynamoDB is a fully managed NoSQL database service
- Designed for high performance and scalability
- Stores data as key-value or document format
- Commonly used in modern cloud applications

Serverless Nature of DynamoDB

- No server management required
- Automatic scaling
- Built-in high availability
- Pay only for the resources used

Primary Keys in DynamoDB

- Partition Key – uniquely identifies an item
- Sort Key – used with partition key for ordering
- Simple Primary Key = Partition Key only
- Composite Key = Partition Key + Sort Key

Item Size and Storage Limits

- Maximum item size = 400 KB
- Includes attribute names and values
- Large files should be stored in object storage services
- DynamoDB should store metadata references instead

Query vs Scan

Query:

- Searches using partition key
- Fast and efficient

Scan:

- Reads the entire table
- Slower and more expensive

Capacity Modes in DynamoDB

Provisioned Capacity:

- User defines read and write capacity units
- Best for predictable workloads

On-Demand Capacity:

- Automatically adjusts capacity
- Best for unpredictable traffic

Local Secondary Index (LSI)

- Uses the same partition key as the main table
- Uses a different sort key
- Must be created during table creation
- Maximum of 5 LSIs per table

Global Secondary Index (GSI)

- Allows different partition and sort keys
- Can be created anytime
- Supports additional query patterns
- Uses separate read/write capacity

Need for Global Secondary Index

- Allows queries using attributes other than the primary key
- Avoids expensive full table scans
- Improves performance and scalability
- Enables multiple access patterns in applications

Configuring AWS CLI with Security Credentials

Set up the AWS CLI to authenticate and authorize DynamoDB operations using IAM security credentials.

PREREQUISITES

STEP 1

Install AWS CLI

Download v2 from AWS official docs for your OS

STEP 2

IAM User + Policy

Create an IAM user with `AmazonDynamoDBFullAccess` policy

STEP 3

Access Keys

Generate Access Key ID and Secret Access Key from IAM console

CREDENTIAL FIELDS

ACCESS KEY ID

```
AKIAIOSFODNN7EXAMPLE
```

SECRET ACCESS KEY

```
wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

DEFAULT REGION

```
us-east-1
```

DEFAULT OUTPUT FORMAT

```
json
```

BASH

```
# Interactive configuration wizard
aws configure

# Output prompts:
AWS Access Key ID [None]:
  AKIAIOSFODNN7EXAMPLE

AWS Secret Access Key [None]:
  wJalrXUtnFEMI/...

Default region name [None]:
  us-east-1

Default output format [None]:
  json
```

NAMED PROFILES (ADVANCED)

```bash
# Create a named profile for DynamoDB dev environment
aws configure --profile dynamo-dev

# Use the named profile in commands
aws dynamodb list-tables --profile dynamo-dev

# Or set as environment variable
export AWS_PROFILE=dynamo-dev

# Verify your identity
aws sts get-caller-identity
```

CREDENTIALS FILE LOCATION

WINDOWS

`C:\Users\USERNAME\.aws\credentials`

`C:\Users\USERNAME\.aws\config`

MACOS / LINUX

`~/.aws/credentials`

`~/.aws/config`

Security Best Practice: Never embed access keys in code or commit them to version control. Use IAM roles for EC2/Lambda environments. Rotate keys regularly and apply least-privilege policies.

Accessing DynamoDB via AWS CLI

Manage tables, explore data, and run operations directly from your terminal using the `aws dynamodb` command group.

ESSENTIAL TABLE COMMANDS

1 **Create a Table**

BASH

```
aws dynamodb create-table \
  --table-name Users \
  --attribute-definitions \
    AttributeName=UserId,AttributeType=S \
    AttributeName=Email,AttributeType=S \
  --key-schema \
    AttributeName=UserId,KeyType=HASH \
    AttributeName=Email,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST
```

2 List & Describe Tables

```bash
# List all tables in the configured region
aws dynamodb list-tables

# Describe a specific table (schema, status, indexes)
aws dynamodb describe-table \
  --table-name Users
```

3 Delete a Table

```bash
aws dynamodb delete-table \
  --table-name Users
```

ATTRIBUTE TYPES REFERENCE

TYPE CODE	FULL TYPE	EXAMPLE
S	String	`"John Doe"`
N	Number	`"42"` or `"3.14"`
B	Binary	Base64-encoded binary data
BOOL	Boolean	`true` / `false`
L	List	`["a", "b", "c"]`
M	Map	`{"key": "value"}`

```bash
# Override region for a single command
aws dynamodb list-tables \
  --region ap-south-1

# Use local DynamoDB for development
aws dynamodb list-tables \
  --endpoint-url http://localhost:8000
```

CRUD Operations via CLI

Create, Read, Update, and Delete items in DynamoDB tables using structured CLI commands with JSON payloads.

CREATE | READ | UPDATE | DELETE | SCAN / QUERY

put-item — Inserts a new item or fully replaces an existing one. Use `ConditionExpression` to prevent overwriting.

BASH

```
aws dynamodb put-item \
  --table-name Users \
  --item '{
    "UserId":  {"S": "user-001"},
    "Email":   {"S": "alice@example.com"},
    "Name":    {"S": "Alice Johnson"},
    "Age":     {"N": "29"},
    "Active":  {"BOOL": true}
  }' \
  --condition-expression \
    "attribute_not_exists(UserId)"
```

get-item — Retrieves a single item by its exact primary key. The fastest, cheapest read operation in DynamoDB.

BASH

```bash
# Get a single item by primary key
aws dynamodb get-item \
  --table-name Users \
  --key '{
    "UserId": {"S": "user-001"},
    "Email":  {"S": "alice@example.com"}
  }' \
  --projection-expression "#n, Age, Active" \
  --expression-attribute-names '{"#n": "Name"}'
```

update-item — Modifies specific attributes without touching the rest. Supports SET, REMOVE, ADD, and DELETE operations atomically.

BASH

```
aws dynamodb update-item \
  --table-name Users \
  --key '{
    "UserId": {"S": "user-001"},
    "Email":  {"S": "alice@example.com"}
  }' \
  --update-expression \
    "SET Age = :newAge, #n = :newName REMOVE OldField" \
  --expression-attribute-values '{
    ":newAge":  {"N": "30"},
    ":newName": {"S": "Alice M. Johnson"}
  }' \
  --expression-attribute-names '{"#n": "Name"}' \
  --return-values ALL_NEW
```

- **delete-item** — Permanently removes an item by primary key. Use `ConditionExpression` to only delete when conditions are met (safe deletes).

BASH

```bash
# Simple delete by primary key
aws dynamodb delete-item \
  --table-name Users \
  --key '{
    "UserId": {"S": "user-001"},
    "Email":  {"S": "alice@example.com"}
  }'

# Conditional delete — only if user is inactive
aws dynamodb delete-item \
  --table-name Users \
  --key '{"UserId": {"S": "user-001"}}' \
  --condition-expression "Active = :f" \
  --expression-attribute-values \
    '{":f": {"BOOL": false}}'
```

query uses the partition key (cheap) — **scan** reads every item (expensive). Always prefer `query` in production.

BASH – QUERY

```bash
# Query all items for a given partition key
aws dynamodb query \
  --table-name Users \
  --key-condition-expression \
    "UserId = :uid" \
  --expression-attribute-values \
    '{":uid": {"S": "user-001"}}' \
  --limit 10
```

BASH – SCAN WITH FILTER

```bash
# Scan full table, filter result by Age > 25
aws dynamodb scan \
  --table-name Users \
  --filter-expression "Age > :minAge" \
  --expression-attribute-values \
    '{":minAge": {"N": "25"}}'
```

Conclusion

- DBMS enables structured data management
- SQL databases handle relational data
- DynamoDB is a scalable NoSQL solution
- Proper table design and indexing improve performance